03 Exercise Notebook 3

February 28, 2026

1 Exercise 3

In this exercise, you will analyse a dataset obtained from the London transport system (TfL). The data is in a filled called `tfl_readership.csv` (comma-separated-values format). As in Exercise 2, we will load and view the data using `pandas`.

[24]:
```
# If you are running this on Google Colab, uncomment and run the following␣
↪lines; otherwise ignore this cell
# from google.colab import drive
# drive.mount('/content/drive')
```

[25]:
```
import math
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd
```

[26]:
```
# Load data
df_tfl = pd.read_csv('tfl_ridership.csv')
# If running on Google Colab change path to '/content/drive/MyDrive/
↪IB-Data-Science/Exercises/tfl_ridership.csv'

df_tfl.head(13)
```

[26]:
```
       Year Period       Start         End Days  Bus cash (000s)  \
0   2000/01   P 01  01 Apr '00  29 Apr '00  29d              884
1   2000/01   P 02  30 Apr '00  27 May '00  28d              949
2   2000/01   P 03  28 May '00  24 Jun '00  28d              945
3   2000/01   P 04  25 Jun '00  22 Jul '00  28d              981
4   2000/01   P 05  23 Jul '00  19 Aug '00  28d              958
5   2000/01   P 06  20 Aug '00  16 Sep '00  28d              984
6   2000/01   P 07  17 Sep '00  14 Oct '00  28d             1001
7   2000/01   P 08  15 Oct '00  11 Nov '00  28d              979
8   2000/01   P 09  12 Nov '00  09 Dec '00  28d              971
9   2000/01   P 10  10 Dec '00  06 Jan '01  28d              912
10  2000/01   P 11  07 Jan '01  03 Feb '01  28d              943
11  2000/01   P 12  04 Feb '01  03 Mar '01  28d              975
12  2000/01   P 13  04 Mar '01  31 Mar '01  28d              974
```

	Bus Oyster PAYG (000s)	Bus Contactless (000s)	\
0	0	0	
1	0	0	
2	0	0	
3	0	0	
4	0	0	
5	0	0	
6	0	0	
7	0	0	
8	0	0	
9	0	0	
10	0	0	
11	0	0	
12	0	0	

	Bus One Day Bus Pass (000s)	Bus Day Travelcard (000s)	…	\
0	210	231	…	
1	214	205	…	
2	209	221	…	
3	216	241	…	
4	225	248	…	
5	243	236	…	
6	205	216	…	
7	199	221	…	
8	184	212	…	
9	192	211	…	
10	193	186	…	
11	194	210	…	
12	186	204	…	

	Tube Contactless (000s)	Tube Day Travelcard (000s)	\
0	0	655	
1	0	605	
2	0	650	
3	0	708	
4	0	730	
5	0	702	
6	0	639	
7	0	668	
8	0	640	
9	0	631	
10	0	556	
11	0	617	
12	0	584	

	Tube Season Travelcard (000s)	Tube Other incl free (000s)	\
0	1066	200	

```
1                           1168                           217
2                           1154                           212
3                           1196                           214
4                           1165                           165
5                           1164                           151
6                           1286                           196
7                           1298                           220
8                           1302                           242
9                            993                           195
10                          1259                           234
11                          1237                           246
12                          1262                           266

    Tube Total (000s)  TfL Rail (000s)  Overground (000s)  DLR (000s)  \
0                2509                0                  0          96
1                2598                0                  0          93
2                2623                0                  0          98
3                2761                0                  0         105
4                2643                0                  0         103
5                2608                0                  0         100
6                2763                0                  0         107
7                2819                0                  0         113
8                2839                0                  0         114
9                2359                0                  0          90
10               2634                0                  0         110
11               2688                0                  0         120
12               2699                0                  0         119

    Tram (000s)  Air Line (000s)
0          45.8             0.0
1          46.5             0.0
2          47.1             0.0
3          50.8             0.0
4          50.3             0.0
5          49.2             0.0
6          48.8             0.0
7          51.5             0.0
8          54.0             0.0
9          55.3             0.0
10         50.1             0.0
11         50.5             0.0
12         47.7             0.0

[13 rows x 26 columns]
```

Each row of our data frame represents the average daily ridership over a 28/29 day period for various types of transport and tickets (bus, tube etc.). We have used the `.head()` command to display the top 13 rows of the data frame (corresponding to one year). Focusing on the "Tube

Total" column, notice the dip in ridership in row 9 (presumably due to Christmas/New Year's), and also the slight dip during the summer (rows 4,5).

[27]:
```
#df_tfl.sample(3)  #random sample of 3 rows
df_tfl.tail(3)  #last 3 rows
```

[27]:
```
        Year Period       Start         End Days  Bus cash (000s)  \
242  2018/19   P 09  11 Nov '18  08 Dec '18  28d                0
243  2018/19   P 10  09 Dec '18  05 Jan '19  28d                0
244  2018/19   P 11  06 Jan '19  02 Feb '19  28d                0

     Bus Oyster PAYG (000s)  Bus Contactless (000s)  \
242                    1110                     1089
243                    1001                      949
244                    1036                     1075

     Bus One Day Bus Pass (000s)  Bus Day Travelcard (000s)  …  \
242                            0                         41  …
243                            0                         38  …
244                            0                         30  …

     Tube Contactless (000s)  Tube Day Travelcard (000s)  \
242                     1399                         249
243                     1110                         242
244                     1310                         204

     Tube Season Travelcard (000s)  Tube Other incl free (000s)  \
242                           1017                          334
243                            632                          259
244                            924                          305

     Tube Total (000s)  TfL Rail (000s)  Overground (000s)  DLR (000s)  \
242               4221              996                557         355
243               3279              750                414         270
244               3809              929                517         333

     Tram (000s)  Air Line (000s)
242         84.1              2.6
243         66.3              3.2
244         79.3              2.3

[3 rows x 26 columns]
```

The dataframe contains $N = 245$ counting periods (of 28/29 days each) from 1 April 2000 to 2 Feb 2019. We now define a numpy array consisting of the values in the ' Tube Total (000s)' column:

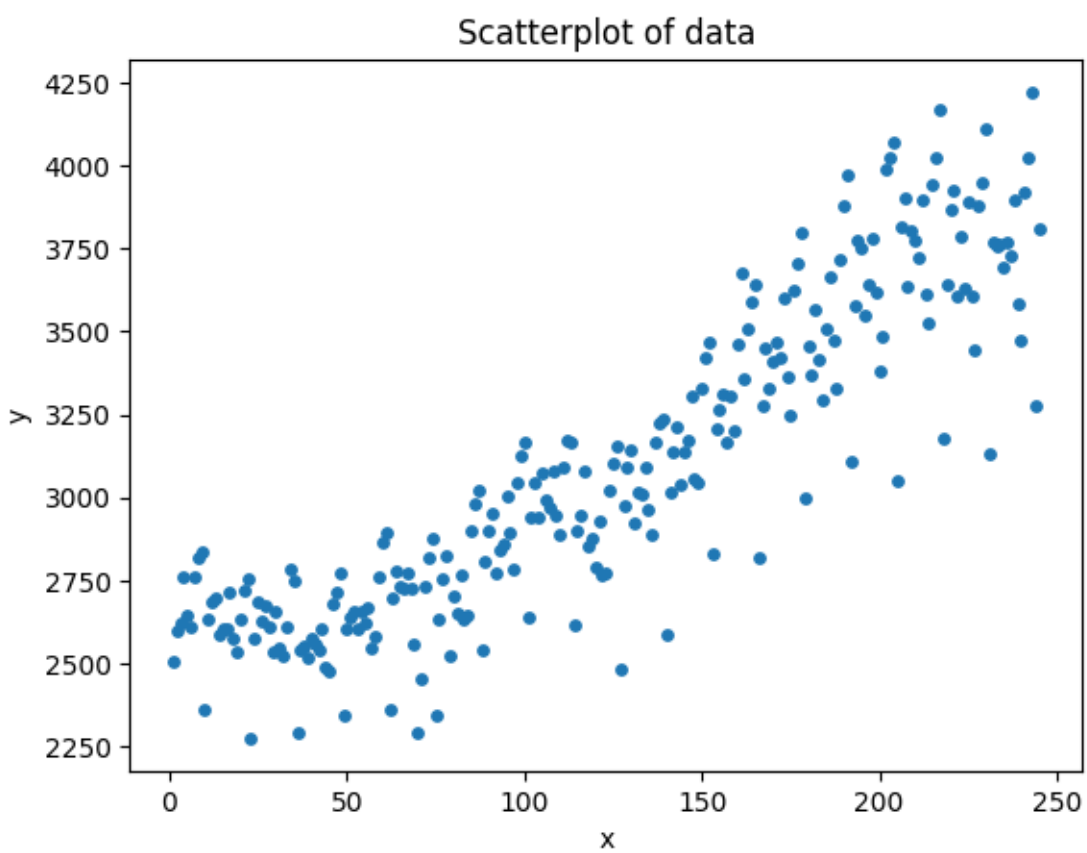
[28]:
```
yvals = np.array(df_tfl['Tube Total (000s)'])
N = np.size(yvals)
```

```
xvals = np.linspace(1,N,N) #an array containing the values 1,2....,N
```

We now have a time series consisting of points (x_i, y_i), for $i = 1, \ldots, N$, where y_i is the average daily tube rideship in counting period $x_i = i$.

1.1 3a) Plot the data in a scatterplot

[29]:
```
#Your code for scatterplot here

import matplotlib.pyplot as plt

x = xvals
y = yvals

plt.scatter(x, y, s=15)
plt.xlabel('x')
plt.ylabel('y')
plt.title('Scatterplot of data')
plt.show()
```



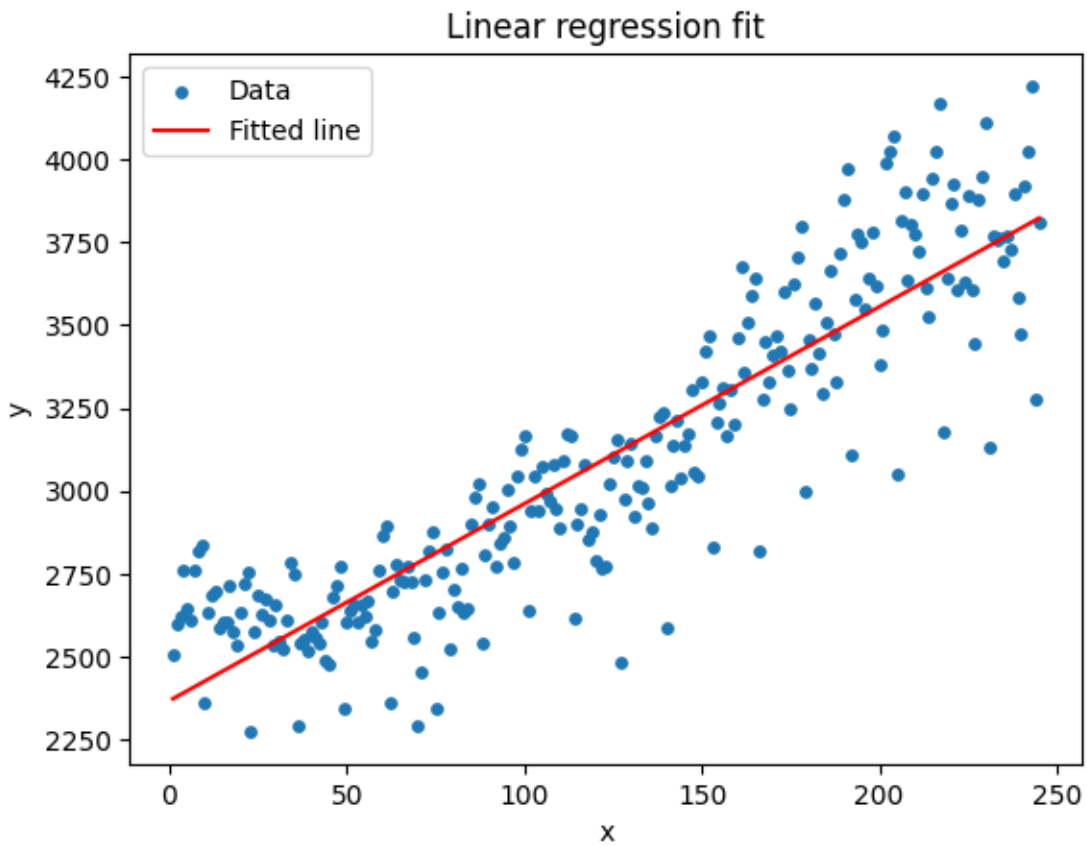
1.2 3b) Fit a linear model $f(x) = \beta_0 + \beta_1 x$ to the data

- Print the values of the regression coefficients β_0, β_1 determined using least-squares.
- Plot the fitted model and the scatterplot on the same plot.
- Compute and print the **MSE** and the R^2 coefficient for the fitted model.

All numerical outputs should be displayed to three decimal places.

[30]:
```
#Your code here

import numpy as np
import matplotlib.pyplot as plt

# Data
x = xvals
y = yvals
N = len(y)

# Design matrix [1  x]
X = np.column_stack((np.ones(N), x))

# Least squares solution
beta = np.linalg.inv(X.T @ X) @ (X.T @ y)
beta0, beta1 = beta

# Fitted values
y_fit = X @ beta

# Compute errors
SSE = np.sum((y - y_fit)**2)
MSE = SSE / N
SST = np.sum((y - np.mean(y))**2)
R2 = 1 - SSE/SST

# Print to 3 decimal places
print(f"beta0 = {beta0:.3f}")
print(f"beta1 = {beta1:.3f}")
print(f"MSE = {MSE:.3f}")
print(f"R^2 = {R2:.3f}")

# Plot
plt.scatter(x, y, s=15, label='Data')
plt.plot(x, y_fit, color='red', label='Fitted line')
plt.xlabel('x')
plt.ylabel('y')
plt.title('Linear regression fit')
plt.legend()
plt.show()
```

```
beta0 = 2367.382
beta1 = 5.939
MSE = 45323.636
R^2 = 0.796
```

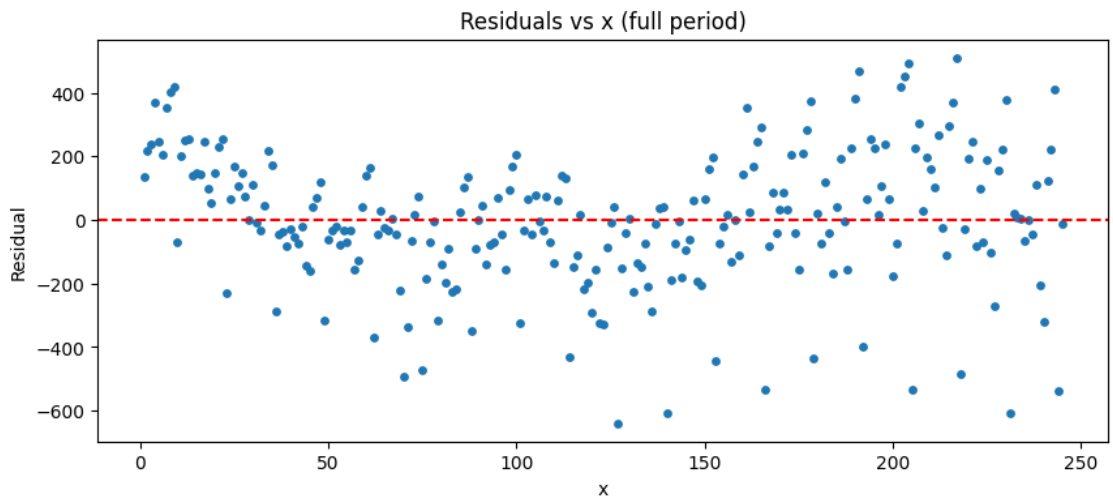


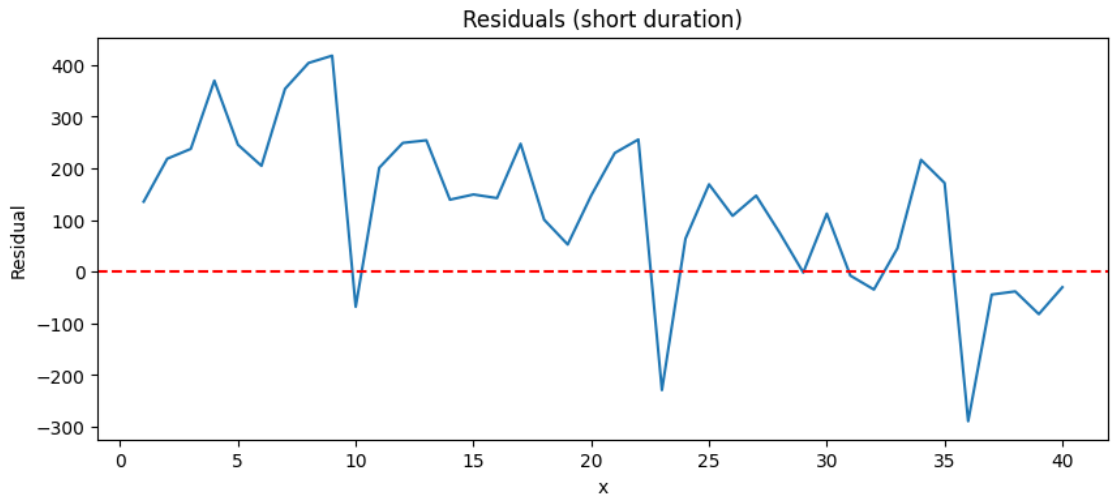
1.3 3c) Plotting the residuals

- Plot the residuals on a scatterplot
- Also plot the residuals over a short duration and comment on whether you can discern any periodic components.

[31]:
```
import numpy as np
import matplotlib.pyplot as plt

# Residuals
residuals = y - y_fit

# --- Full residual plot ---
plt.figure(figsize=(10,4))
plt.scatter(x, residuals, s=15)
```

```
plt.axhline(0, color='red', linestyle='--')
plt.xlabel('x')
plt.ylabel('Residual')
plt.title('Residuals vs x (full period)')
plt.show()

# --- Short duration (e.g. first 40 points) ---
short_n = 40

plt.figure(figsize=(10,4))
plt.plot(x[:short_n], residuals[:short_n])
plt.axhline(0, color='red', linestyle='--')
plt.xlabel('x')
plt.ylabel('Residual')
plt.title('Residuals (short duration)')
plt.show()
```



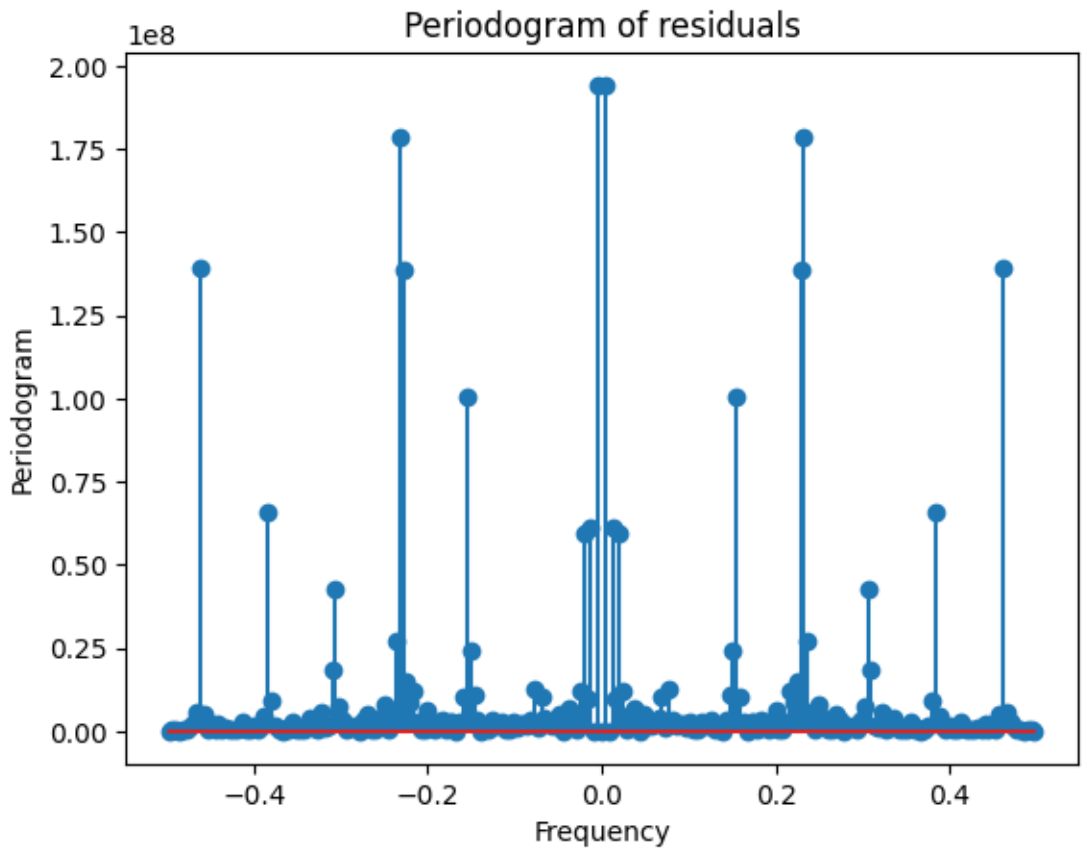


Residuals exhibit a oscillatory pattern with possibly more than one characteristic timescale, indicating periodic components not captured by the linear trend.

1.3.1 3d) Periodogram

- Compute and plot the peridogram of the residuals. (Recall that the periodogram is the squared-magnitude of the DFT coefficients.)
- Identify the indices/frequencies for which the periogram value exceeds **50%** of the maximum.

[32]:
```
import numpy as np
import matplotlib.pyplot as plt

# residuals
N = len(residuals)

# DFT
X = np.fft.fft(residuals)

# Periodogram = squared magnitude
P = np.abs(X)**2

freq = np.fft.fftfreq(N, d=1)

plt.figure()
plt.stem(freq, P)
plt.xlabel('Frequency')
plt.ylabel('Periodogram')
plt.title('Periodogram of residuals')
plt.show()
```



[33]:
```
threshold = 0.5 * np.max(P)

indices = np.where(P > threshold)[0]
frequencies = freq[indices]

print("Indices exceeding 50% of max:")
print(indices)

print("\nCorresponding frequencies:")
print(frequencies)
```

```
Indices exceeding 50% of max:
[  1  38  56  57 113 132 188 189 207 244]

Corresponding frequencies:
[ 0.00408163  0.15510204  0.22857143  0.23265306  0.46122449 -0.46122449
 -0.23265306 -0.22857143 -0.15510204 -0.00408163]
```

1.4 3e) To the residuals, fit a model of the form

$$\beta_{1s}\sin(\omega_1 x) + \beta_{1c}\cos(\omega_1 x) + \beta_{2s}\sin(\omega_2 x) + \beta_{2c}\cos(\omega_2 x) + ... + \beta_{Ks}\sin(\omega_K x) + \beta_{Kc}\cos(\omega_K x).$$

The frequencies $\omega_1, \ldots, \omega_K$ in the model are those corresponding to the indices identified in Part 2c. (Hint: Each of the sines and cosines will correspond to one column in your X-matrix.)

- Print the values of the regression coefficients obtained using least-squares.

All numerical outputs should be displayed to three decimal places.

[34]:
```
# keep only positive frequencies
pos_freqs = frequencies[frequencies > 0]

# convert to angular frequencies
omega = 2 * np.pi * pos_freqs

N = len(residuals)
x = np.arange(N)

# build design matrix
X = []

for w in omega:
    X.append(np.sin(w * x))
    X.append(np.cos(w * x))
X = np.column_stack(X)

# Compute beta
XT = X.T
beta = np.linalg.inv(XT @ X) @ XT @ residuals
print("Regression coefficients:")
print(np.round(beta, 3))
```

```
Regression coefficients:
[-53.84  100.208  79.294  20.666  91.848 -28.165 -63.069  89.048 -53.366
 -80.08 ]
```

1.4.1 3f) The combined fit

- Plot the combined fit together with a scatterplot of the data
- Compute and print the final **MSE** and R^2 coefficient. Comment on the improvement over the linear fit.

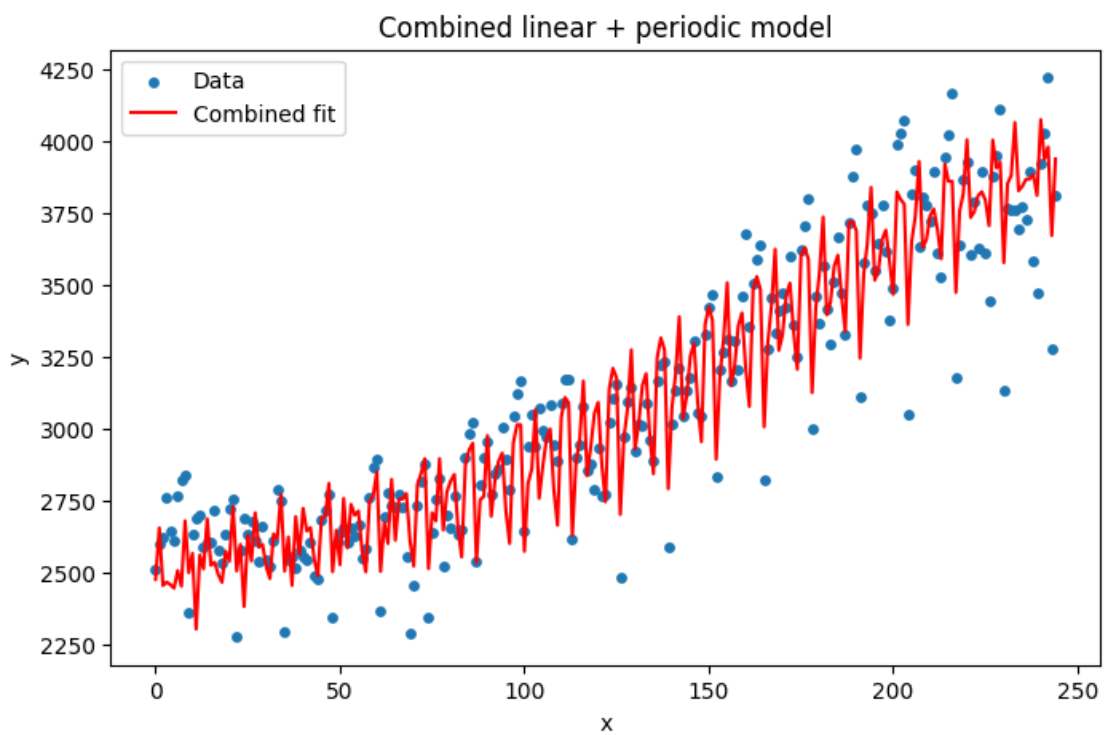
The combined fit, which corresponds to the full model

$$f(x) = \beta_0 + \beta_1 x + \beta_{s1} \sin(\omega_1 x) + \beta_{c1} \cos(\omega_1 x) + \ldots + \beta_{sk} \sin(\omega_k x) + \beta_{ck} \cos(\omega_k x),$$

can be obtained by adding the fits in parts 2b) and 2e).

[35]:
```
y_linear = y_fit
y_periodic = X @ beta
# Combined fit
y_combined = y_linear + y_periodic
```

```python
# Plot
plt.figure(figsize=(8,5))
plt.scatter(x, yvals, s=15, label="Data")
plt.plot(x, y_combined, color='red', label="Combined fit")
plt.xlabel("x")
plt.ylabel("y")
plt.legend()
plt.title("Combined linear + periodic model")
plt.show()

# Compute metrics
res_combined = yvals - y_combined

SSE_comb = np.sum(res_combined**2)
MSE_comb = SSE_comb / len(yvals)

SST = np.sum((yvals - np.mean(yvals))**2)
R2_comb = 1 - SSE_comb / SST

print(f"MSE (combined) = {MSE_comb:.3f}")
print(f"R^2 (combined) = {R2_comb:.3f}")
```

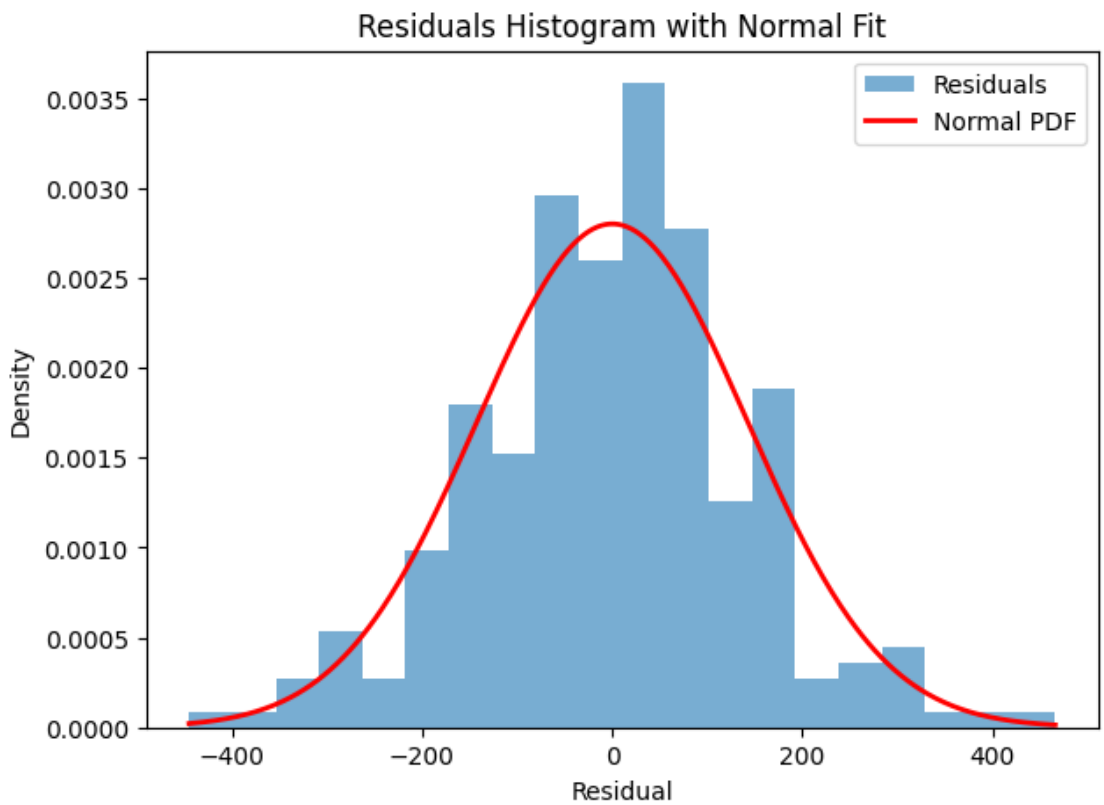


```
MSE (combined) = 20297.501
R^2 (combined) = 0.908
```

The combined model reduces the MSE compared to the linear model and increases the R^2. This indicates that the periodic components explain structured variation that the linear trend alone cannot capture.

Extra: Plotting the Histogram of Res and Comparing with Normal Distribution

[36]:
```python
import numpy as np
import matplotlib.pyplot as plt
from scipy.stats import norm

# residuals from combined model
res = res_combined   # make sure this matches your variable name

# Compute mean and std
mu = np.mean(res)
sigma = np.std(res)

# Histogram (density=True normalizes area to 1)
plt.figure(figsize=(7,5))
plt.hist(res, bins=20, density=True, alpha=0.6, label="Residuals")

# Normal PDF
x_vals = np.linspace(min(res), max(res), 400)
normal_pdf = norm.pdf(x_vals, mu, sigma)

plt.plot(x_vals, normal_pdf, 'r', linewidth=2, label="Normal PDF")

plt.xlabel("Residual")
plt.ylabel("Density")
plt.title("Residuals Histogram with Normal Fit")
plt.legend()
plt.show()

print(f"Mean = {mu:.3f}")
print(f"Std Dev = {sigma:.3f}")
```




```
Mean = 0.000
Std Dev = 142.469
```